

01

A note for existing CodePush customers

From CodePush to Patch

We still host OTA for you. Patch is the dedicated product we built next.

What's happening

CodePush is moving to Patch. Dedicated product, own dashboard and CLI, not a new screen in the CI/CD UI.

Hosted **Same operator, new service**

Codemagic runs Patch. You get a dashboard login, not a box to operate. CodePush stays up until old binaries are gone.

Client **New SDK and protocol**

The CodePush client cannot take Patch updates. `@codemagic/react-native-patch` has to ship in a native binary.

Model **The release shape stays**

Apps per platform. Staging and Production. `sync()` install modes. `release-react`, mandatory, rollout, target binary version.

What you need to do, and when

Nothing switches off on its own. The required action is one native release that includes the Patch client. OTA cannot deliver that change.

Now **Keep shipping on CodePush**

We provision Patch, mirror apps and deployments, and hand you dashboard access plus `cm_pat_` tokens. Existing binaries keep updating as they do today.

You **Ship a native build with Patch**

Replace the SDK, point CI at `cmpatch release-react`, and ship that binary through the store or your internal channel. That binary is the cutover.

After **Drain old binaries**

Publish OTA to the new binary. Leave CodePush running until the old client is gone from the fleet.

Why this is better

You already ship OTA with us. Patch is the product around that job: a dashboard you can run releases from, fingerprinting that ties JS to a binary, and a CLI your CI can reuse.

Dashboard

Releases you can actually run from the UI

Manage apps, promote Staging to Production, and watch downloads, installs, and failures. Team access is RBAC in the dashboard, not a pile of access keys.

Fingerprint

Stop shipping JS to the wrong binary

On every release, the CLI hashes the native project and ties the bundle to a binary version. That pair is what keeps an update off an incompatible build.

CLI

A release command your CI can trust

`cmpatch release-react` keeps the habit you already have: mandatory, rollout, target binary version. Identity is OAuth, tokens are `cm_pat_`, not a long-lived access key in a wiki.

Step by step

We stand up the hosted Patch server. You swap the client in a native release, then point CI at the new CLI.

01 We provision Patch

Same vendor, new service. Apps and deployments can mirror your CodePush layout. You get dashboard login and CLI tokens.

02 Replace the SDK

Drop the CodePush client. Add `@codemagic/react-native-patch`. Call `sync()` from app entry. This has to ship in a store or internal binary.

03 Point CI at cmpatch

Same release-react shape, new CLI and `cm_pat_` tokens. Keep Staging ahead of Production.

04 Cut over

Ship the native build. Publish OTA to that binary. Leave CodePush running until old binaries are gone.

06

Walk through it with us

If you already run CodePush with us, this is a planned cutover. We keep operating the server. Your team ships one native release.

[Talk to sales](#)[Migration guide](#)

[Docs](#) [GitHub](#) [Pricing](#) [© Nevercode Ltd.](#)